That’s all well and good, but how should we interpret these differences? When should we choose one over the other?

I like to think of middleware versus filters as a question of specificity. Middleware is the more general concept, operating on lower-level primitives such as `HttpContext`, so it has wider reach. If the functionality you need has no endpoint-specific requirements, you should use a middleware component. Exception handling is a great example; exceptions could happen anywhere in your application, and you need to handle them, so using exception-handling middleware makes sense.

On the other hand, if you *do* need access to endpoint details, or if you want to behave differently for some requests, you should consider using a filter. Validation is a good example. Not all requests need the same validation. Requests for static files, for example, don’t need parameter validation, the way requests to an API endpoint do. Applying validation to the endpoints via filters makes sense in this case.

TIP Where possible, consider using middleware for cross-cutting concerns. Use filters when you need different behavior for different endpoints or where the functionality relies on endpoint concepts such as `IResult` objects.

So far, the filters we’ve looked at have been specific to a single endpoint. In section 5.4.3 we look at creating generic filters that you can apply to multiple endpoints.

5.4.3 Generalizing your endpoint filters

One common problem with filters is that they end up closely tied to the *implementation* of your endpoint handlers. Listing 5.9, for example, assumes that the `id` parameter is the first parameter in the method. In this section